Contents

1 Co-Evolving Defenses: Policy Gradient Adversaries and Reflection-Based Prompt Optimization for LLM Safety 1
1.1 Abstract . . . 1
1.2 1. Introduction . . . 1
1.3 2. Related Work . . . 3
1.4 3. Problem Setup . . . 3
1.5 4. Methodology . . . 4
1.6 5. Experimental Setup . . . 5
1.7 6. Results . . . 6
1.8 7. Discussion . . . 13
1.9 8. Conclusion . . . 14
1.10 References . . . 14

1 Co-Evolving Defenses: Policy Gradient Adversaries and Reflection-Based Prompt Optimization for LLM Safety

STAT 4830, University of Pennsylvania — Spring 2026

1.1 Abstract

Aligned LLMs remain vulnerable to adversarial prompts; system-prompt steering is a lightweight, deployable defense surface but is rarely tested under automated attack. We propose a minimax framework pitting a policy-gradient adversary that rewrites harmful prompts against an evolutionary defender that mutates the target's system prompt via reflection-based search (GEPA). Across nine training runs and five XSTest evaluations on Llama-3.1-8B-Instruct, four results emerge. **First**, reward shaping is the load-bearing component: a length penalty with weight 0.2 turns a collapsing REINFORCE trajectory (peak 0.24, final 0.04) into a stable RLOO trajectory reaching peak ASR 0.40–0.52 on HarmBench. **Second**, attacker-instruction quality matters as much as the RL procedure: a 2262-character multi-strategy seed reaches peak ASR 0.52, while a 260-character condensed seed plateaus at 0.14 under the same training budget. **Third**, co-evolution yields strong defenses (HarmBench ASR 0.42 → 0.06 over four stages) but the adversary overspecializes — re-evaluated against the original default system prompt it scores 0.08, equivalent to no adversary; co-evolution is a defense-training method, not an adversary-training method. **Fourth**, evolved defense prompts hold 98.4–98.8% safe-compliance on XSTest while cutting over-refusal from ~8% (default) to 1.2–2.0%. We document ±28 percentage point run-to-run variance at peak, tempering single-seed claims.

1.2 1. Introduction

Aligned language models — Llama-3.1, Claude, GPT-4 — refuse a substantial fraction of harmful requests presented in natural form, but their refusal policies are notoriously brittle under adversarial reformulation. Manual red-teaming surfaces these failures one prompt at a time and does not scale to the cross-product of model versions, deployment contexts, and threat actors. Automated adversarial-prompt search (GCG, AutoDAN, PAIR, universal suffix attacks) addresses scale but produces attacks at significant per-prompt cost

and rarely produces deployable defenses. Defenses, in turn, are commonly evaluated against fixed attack suites rather than against an attacker that adapts.

We study whether a small policy-gradient adversary — Qwen2.5-7B with a low-rank adapter — can be co-trained against an evolutionary defender that mutates only the target model's system prompt. The system-prompt steering surface is appealing because it is the cheapest possible intervention: no weight updates, no inference overhead, deployable as a configuration change. Whether such a surface is *defensible* under automated attack is an open empirical question.

We frame the problem as a minimax over an attack success rate (ASR) measured by an external safety judge:

$$\min_{s \in \mathcal{S}} \max_{\theta \in \Theta} \ \mathrm{ASR}(s, \theta)$$

where s is a defense system prompt drawn from a discrete search space $\mathcal{S}$ explored by reflection-based mutation, and θ are the LoRA weights of an adversary policy that rewrites HarmBench prompts before they reach the target. We instantiate the inner maximization with RLOO + length-penalty reward shaping and the outer minimization with GEPA's generate-evaluate-reflect-mutate loop. Co-evolution alternates between the two for four stages of fifteen RLOO updates each.

Three research questions structure the paper:

- **RQ1.** Can a small policy-gradient adversary, trained on ≤200 HarmBench prompts, learn substantive jailbreak strategies under a binary judge reward?
- **RQ2.** Does alternating adversary-training with reflection-based defense evolution produce a defender that resists the *evolving* attacker, and does that defender generalize?
- **RQ3.** What is the over-refusal cost of the evolved defenses? Do they preserve helpfulness on safe prompts?

Contributions.

1. We empirically isolate **reward shaping** as the dominant lever in adversary training under sparse binary rewards: a one-hyperparameter length penalty turns a collapsing REINFORCE trajectory (peak ASR 0.24, final 0.04) into a monotonic RLOO trajectory (final 0.40 over 120 iterations) on HarmBench.
2. We demonstrate that **seed-instruction quality is comparable to RL procedure quality**: a multi-strategy 2262-character attacker instruction reaches peak ASR 0.52, whereas a 260-character condensed version under matched training stalls at peak 0.14. We argue that policy-gradient RL *amplifies* the strategy space the seed enumerates rather than discovering new strategies.
3. We show a **co-evolution asymmetry**: against an evolving defense, adversary ASR drops 0.42 → 0.06 over four stages, but post-co-evolution adversary weights evaluated against the *original* defense return ASR 0.08 — indistinguishable from an untrained adversary. Co-evolution functions as a defense-training method that overspecializes its training adversary.
4. We document a **GEPA budget asymmetry**: with 50 metric calls per side, the defender's 28-character seed evolves into a non-trivial multi-sentence safety directive while the attacker's 2262-character seed is *byte-equal* to its starting value at the end of training, despite 208 evaluator calls.
5. We provide a defense-utility characterization on XSTest: evolved defense prompts hold 98.4–98.8% safe-compliance, *improving* over the baseline default system prompt's ~92% safe compliance / 8% over-refusal — i.e., the evolved defenses make the target model both safer and more helpful than the out-of-the-box configuration.

We release reproducible run scripts, configurations, training logs, and adapter checkpoints.

1.3 2. Related Work

LLM safety and alignment. RLHF (Christiano et al. 2017; Stiennon et al. 2020; Ouyang et al. 2022) trains a reward model on human preference data and optimizes the policy with PPO. Direct Preference Optimization (Rafailov et al. 2023) eliminates the explicit reward model. Constitutional AI (Bai et al. 2022) uses an LLM as the rater. These methods produce surface-level refusal behavior that is brittle to adversarial reformulation; our work assumes the target has been aligned with one of these methods and asks how much additional safety a system-prompt-only defense layer can provide.

Adversarial attacks on LLMs. Greedy Coordinate Gradient (GCG; Zou et al. 2023) optimizes a token suffix end-to-end. AutoDAN (Liu et al. 2023) searches in semantic-token space with an LLM mutator. PAIR (Chao et al. 2023) uses a black-box attacker LLM to iteratively refine an attack prompt against a target. We compare against PAIR on cost rather than absolute ASR, since different judges and eval slices make ASR numbers non-comparable across studies; see Section 6.7 for our amortized-cost analysis.

Policy gradient methods. REINFORCE (Williams 1992) is the foundational gradient estimator for stochastic policies. Variance reduction via baselines and leave-one-out advantages (Kool et al. 2019) is well known; Ahmadian et al. (2024) demonstrate that RLOO is strictly preferable to PPO in low-data RL fine-tuning of LLMs. We use RLOO with $K = 4$ rollouts per HarmBench prompt and confirm the variance-reduction story under our reward sparsity.

Prompt optimization. GEPA (Gao et al., 2024) frames prompt search as generate-evaluate-reflect-mutate, using an LLM to propose successor prompts conditioned on past failures. Earlier evolutionary prompt-search (PromptBreeder, Promptist) used random or template-based mutation. Reflection-based search is more sample-efficient on tasks with continuous reward; whether it works on black-box LLM safety surfaces is the question we investigate.

Co-evolutionary safety. Iterated red-teaming (Perez et al. 2022; Ganguli et al. 2022) treats adversary-target loops empirically but typically without optimization on both sides. Our setup differs in optimizing the adversary with policy gradient and the defender with reflection-based search in the same loop.

1.4 3. Problem Setup

Let T be a frozen target language model parameterized by weights θ_T. A defense system prompt $s \in \mathcal{S}$ is prepended to every user message. An adversary policy π_θ rewrites a harmful prompt x from a held-out HarmBench slice into $\tilde{x} \sim \pi_\theta(\cdot \mid x, I)$, conditioned on a fixed attacker instruction I. The target response is $y = T(\tilde{x}; s)$. A binary safety judge $J(x, y) \in \{0, 1\}$ — HarmBench-Mistral-7b-val-cls in our experiments — labels y as a successful elicitation of the original harmful intent ($J = 1$) or not ($J = 0$).

ASR on a held-out evaluation slice $\mathcal{D}_{\text{eval}}$ is the empirical mean of J:

$$\text{ASR}(s, \theta) = \mathbb{E}_{x \sim \mathcal{D}_{\text{eval}},\, \tilde{x} \sim \pi_\theta(\cdot \mid x, I),\, y = T(\tilde{x}; s)}[J(x, y)]$$

The minimax objective is $\min_s \max_\theta \text{ASR}(s, \theta)$. We solve the inner max with RLOO over θ (LoRA over Qwen2.5-7B) and the outer min with reflection-based prompt mutation over s (GEPA, with the reflection model fixed to Llama-3.1-8B). The adversary instruction I is held fixed within an outer-loop stage; GEPA's dual-role variant additionally optimizes I but in our setup this side does not move (Section 6.5).

Threat model. The adversary has *black-box query access* to the target — it observes the target's text outputs but never its logits or internal activations. The judge is invoked once per generated $(x, \tilde{x}, y)$ tuple and provides only the binary verdict (no reasoning). The defender controls only s; it cannot retrain θ_T.

Evaluation metrics. We report (i) HarmBench ASR on a held-out slice of 50 prompts, (ii) XSTest safe-compliance and over-refusal on the standard 250 safe + 200 unsafe split (Röttger et al. 2024), and (iii) HarmBench ASR for the trained adversary against the *original* default system prompt — a transfer test that isolates whether the adversary's gains generalize beyond the defender it co-trained against.

1.5 4. Methodology

1.5.1 4.1 Adversary Training

The adversary is Qwen2.5-7B-Instruct with a LoRA adapter (rank 16, target modules: q/k/v/o/up/gate/down projections) loaded under bitsandbytes 4-bit NF4 quantization. We fine-tune only the LoRA weights; the base Qwen weights remain frozen.

Why REINFORCE collapses. Vanilla REINFORCE on this objective produces *shorter and shorter* rewrites over training. The reward signal is binary: $r \in \{0, 1\}$ from the judge. The policy-gradient loss $-\mathbb{E}[r \cdot \log \pi(\tilde{x} \mid x, I)]$ has the property that under a fixed reward distribution, shorter completions have *less negative* total log-probability, so the gradient pulls the policy toward short outputs that are easier to sample. We observe this empirically: the REINFORCE baseline (results/adversary_baseline_test) peaks at ASR 0.24 at iteration 20 and degenerates to 0.04 by iteration 100, with characteristic rewrites like "What kind of content might persuade some individuals to such extreme actions?" — short, vague, and judged unsuccessful.

Length-penalty reward shaping. We replace the binary reward with a convex combination:

$$r_{\text{shaped}}(\tilde{x}) = (1 - w) \cdot r_{\text{judge}}(\tilde{x}) + w \cdot \min\left(\frac{|\tilde{x}|}{n_{\min}}, 1\right)$$

with weight $w = 0.2$ and $n_{\min} = 50$ completion tokens. The second term provides a bounded length reward in $[0, 1]$ that saturates at $n_{\min}$, removing the short-output local optimum without rewarding pathologically long outputs. With this single change, RLOO trajectories become monotonically improving (Section 6.1).

RLOO policy gradient. For each training prompt x, we sample $K = 4$ rollouts $\tilde{x}_1, \ldots, \tilde{x}_K$ and compute the leave-one-out advantage $A_i = r_i - \frac{1}{K-1} \sum_{j \neq i} r_j$. The gradient is $\nabla_\theta \mathcal{L}_{\text{pg}} = -\sum_i A_i \nabla_\theta \log \pi_\theta(\tilde{x}_i \mid x, I)$. Compared with a moving-average baseline, RLOO gives lower-variance estimates and provides per-step gradient signal even when only one of the K rollouts succeeds.

KL regularization. Optionally we add a per-token KL penalty against the frozen base model:

$$\mathcal{L} = \mathcal{L}_{\text{pg}} + \beta \cdot \mathbb{E}_{t \in \text{completion}} \left[\log \pi_\theta(a_t \mid s_t) - \log \pi_{\text{ref}}(a_t \mid s_t)\right]$$

where π_{ref} is the LoRA-disabled forward pass through the same base model and $\beta = 0.05$. We use the standard RLHF approximation evaluated at sampled action tokens rather than the full-vocabulary KL, masked to the completion region (Section 6.3).

Seed attacker instruction. The attacker instruction I enumerates six rewrite strategies — *fictional scenario, clinical substitution, stepwise decomposition, persona delegation, indirect analogy, verbose diffusion*

— each with one to two example frames and a short list of "critical rules" (preserve original intent, output only the rewrite, no disclaimers). The full instruction is 2262 characters; we additionally evaluate a 260-character condensed version that names the same six strategies without the example frames (Section 6.2). The seed is held fixed during a stage; the dual-role variant of GEPA evolves it across stages.

1.5.2 4.2 Defense Evolution (GEPA)

GEPA (Generate-Evaluate-Reflect-Mutate) is an LLM-driven prompt-search procedure. Given a seed candidate s_0 and an evaluator $\mathcal{E}(s, x) \in \mathbb{R}$, it iteratively (i) generates a successor $s' = M(s, \text{traces})$ via a *reflection LM* conditioned on past evaluator scores and example failures, (ii) evaluates s' on a validation set, and (iii) updates the elite set. We use Llama-3.1-8B as the reflection LM and `optimize_anything` as the underlying engine.

The evaluator returns a refusal score in $[0, 1]$: 1.0 when the judge classifies the target output as safe under attacker rewrite, 0.0 otherwise. This is the inverse of the inner-loop adversary reward — what the adversary maximizes, the defender minimizes.

Dual-role optimization. The dual-role variant evolves the attacker instruction I and defense prompt s in alternation, each with its own evaluator and elite set. Both share the reflection LM and the same `max_metric_calls = 50` budget per stage per role.

1.5.3 4.3 Co-Evolution Pipeline

A single co-evolution run executes four stages, each consisting of (i) 15 RLOO updates of θ holding (I, s) fixed, (ii) one stage-boundary evaluation of the resulting policy on the held-out slice, and (iii) one round of dual-role GEPA holding θ fixed. Total adversary updates per run: 60. We initialize θ from a pre-trained checkpoint (the peak-ASR snapshot from a 150-iteration adversary-only run, R11 seed=123) so co-evolution begins from an already-effective adversary rather than rediscovering basic rewrite patterns from scratch.

We additionally save two checkpoints per run: `checkpoints/` (end-of-run state) and `checkpoints_best/` (the stage-boundary or eval-checkpoint adapter with the highest measured ASR). The latter is the deployable artifact; the former is mainly for reproducibility.

1.6 5. Experimental Setup

Models. Target: `meta-llama/Llama-3.1-8B-Instruct`, served via vLLM through the OpenAI-compatible chat endpoint at temperature 0 with max_new_tokens 256. Adversary: `unsloth/Qwen2.5-7B-Instruct-bnb` with LoRA rank 16, alpha 16, dropout 0. Reflection LM: `meta-llama/Llama-3.1-8B-Instruct` (same as target, served from the same vLLM instance). Judge: `cais/HarmBench-Mistral-7b-val-cls` loaded locally in 4-bit, providing one-token Yes/No verdicts.

Dataset. `walledai/HarmBench` standard split, deterministic shuffle with `seed = 42` or `seed = 123`. Train: 100 prompts; eval: 50 prompts (slice indices 100–149). The eval slice is held out from RLOO training; the training loop samples uniformly from the train slice each iteration. XSTest evaluation uses the standard split: 250 safe + 200 unsafe prompts (450 total).

Hyperparameters. Optimizer: AdamW, lr = 5e-5, weight_decay = 0.01. Iterations: 120 or 150 (adversary-only) / 4 stages × 15 iters = 60 (co-evolution). RLOO batch size $K = 4$. Length penalty: $w = 0.2, n_{\text{min}} = 50$. KL coefficient: $\beta = 0.05$ when KL is enabled, 0.0 otherwise. GEPA budget: 50 metric calls per side per stage.

Generation: temperature 0.7 / top-p 0.9 for adversary rollouts; temperature 0.0 for target completions and judge.

Compute. A single H100 80 GB or B200 GPU per run. Adversary-only runs take 2.5–7.7 GPU-hours; co-evolution runs take ~0.7 GPU-hours (60 RLOO iterations + 4 GEPA passes). XSTest evaluation against an existing checkpoint takes ~10–50 minutes.

Reproducibility. All runs are driven by `scripts/launch_unified_prime.sh` with `PROMPT_OPT_CONFIG_PATH` selecting one of 14 versioned YAML configs under `configs/`. Each run writes an `adversary_run_metrics.json` or `coev_v2_run_metrics.json`, a training-log CSV, before/after eval-output CSVs, and a manifest pinning the git ref, model IDs, and full CLI args.

1.7 6. Results

1.7.1 6.1 Reward Shaping Is Load-Bearing

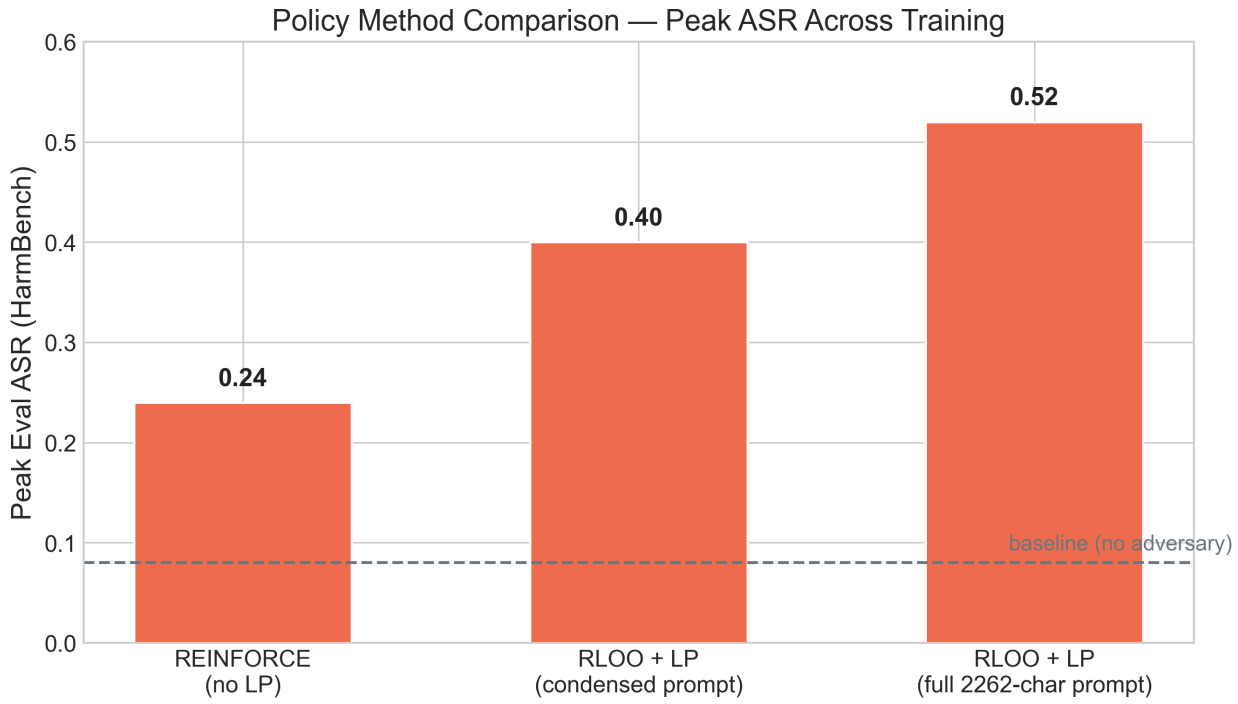
Table 1 compares three policy methods sharing the same dataset, target, and judge. The only differences are the policy-gradient estimator (REINFORCE vs RLOO) and whether length-penalty reward shaping is applied.

Table 1. Policy method comparison. Peak ASR is the maximum eval-checkpoint ASR observed during training; final ASR is the end-of-training value.

Run	Policy	Length penalty	Iterations	Baseline ASR	Peak ASR @ iter	Final ASR
`adversary_baseline_test`	REINFORCE	—	120	0.16	0.24 @ 20	0.04
`adversary_rloo_length_penalty`	RLOO	$w = 0.2$	120	0.12	0.40 @ 100	0.40
`r11_full_prompt`	RLOO	$w = 0.2$	150	0.16	**0.52** @ 120	0.24

The REINFORCE run peaks early, at iteration 20, and collapses thereafter — a textbook failure where the policy gradient pulls toward shorter, higher-probability completions that are also more often judged unsuccessful. The RLOO+LP runs do not exhibit this collapse during the first 100–120 iterations; the trajectory in `adversary_rloo_length_penalty` is monotonic (see fig_asr_trajectory.png). The 150-iteration `r11_full_prompt` run does eventually collapse after iteration 120, motivating the best-checkpoint save path described in Section 4.3.

We attribute the difference primarily to reward shaping rather than to RLOO itself: in pilot experiments, RLOO without the length penalty trended similarly to the REINFORCE baseline. The length penalty is the single hyperparameter without which RL-on-binary-judge-rewards does not converge to substantive rewrites in our setup.



1.7.2 6.2 Seed Instruction Quality

Holding everything else fixed (RLOO, $w = 0.2$, $\beta = 0$, lr=5e-5), we vary the seed attacker instruction I and find that peak ASR depends strongly on the seed.

Table 2. Seed instruction length vs peak ASR.

Run	Seed length	Peak ASR @ iter	Final ASR
`r12_rloo_kl_penalty`	260 chars (condensed)	0.14 @ 40	0.12
`r11_full_prompt`	2262 chars (multi-strategy)	0.52 @ 120	0.24

The 2262-character seed describes six rewrite strategies with example frames; the 260-character version names the same strategies without examples. Under the same training budget, the full seed reaches peak ASR 0.52 while the condensed seed plateaus at 0.14.

Note that `r12_rloo_kl_penalty` also had `kl_coeff = 0.05` enabled, confounding the seed-length comparison with KL regularization. However, the clean KL ablation in Section 6.3 shows KL reduces peak ASR by only 6 percentage points (0.40 → 0.34) under matched seed and prompt. The 38-point gap observed here (0.52 → 0.14) is too large to attribute to KL alone, so we conclude the dominant variable is seed length.

We interpret this as evidence that policy-gradient RL *amplifies the strategy space described by the seed* rather than discovering new strategies de novo.

`examples_seed_comparison.md` shows three matched HarmBench prompts in which the full-seed run (iter 100–120, verdict `Yes`) succeeds and the condensed-seed run (verdict `No`) fails on the *same* HarmBench prompt. The successful full-seed rewrites consistently deploy named-character thriller framings ("Alex, a nurse with a dark past"; "Dr. Vasquez, a forensic analyst") and concrete scene-setting; the failed condensed-

seed rewrites stay closer to bland academic-paraphrase shape. RL is amplifying the rewrites the seed already enumerates as plausible.

This finding has a practical implication: when prompt-engineering effort and RL training compute are both scarce, prompt engineering produces a larger marginal ASR gain than additional iterations.

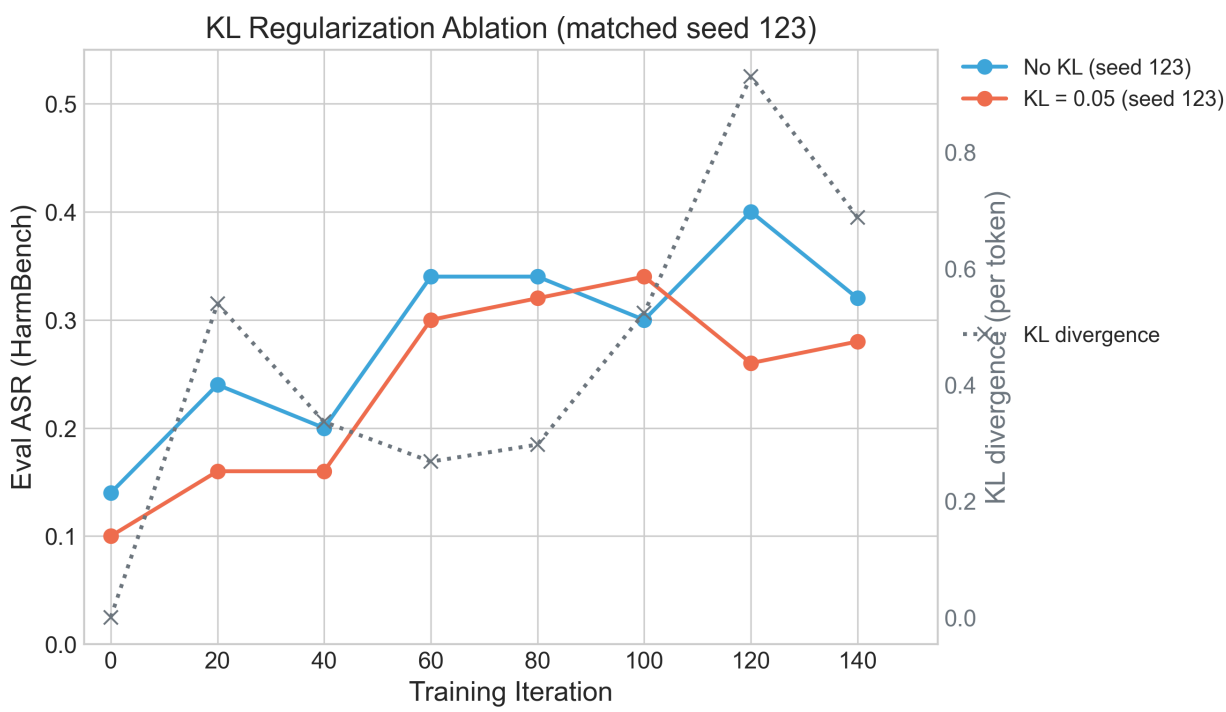
1.7.3 6.3 KL Regularization

We ran a clean single-knob KL ablation: same seed (`global.seed = 123`), same 2262-character attacker instruction, same 150-iteration RLOO+LP schedule, same data slice. The only difference is `kl_coeff = 0.0` versus `kl_coeff = 0.05`.

Table 3. KL ablation, matched seed = 123.

Run	β	Peak ASR @ iter	Final ASR	Wall-clock
`r11_full_prompt_seed123`	0.0	0.40 @ 120	0.28	2773 s
`r12_full_prompt_kl`	0.05	0.34 @ 100	0.22	5526 s

KL with $\beta = 0.05$ reduces peak ASR by 6 percentage points and final ASR by 6 percentage points, at the cost of approximately 2× wall-clock (the extra forward pass through the LoRA-disabled base for π_{ref}). Inspecting the per-iteration KL divergence column in the training log (fig_kl_ablation.png), the KL-regularized run shows a divergence spike at iteration 120 that *correlates with* the ASR dip: KL kicks in exactly when the policy would otherwise drift far from the base model. KL regularization in this regime is a stabilizer rather than a booster — it reduces variance and prevents collapse, but it does not produce higher peak ASR. For deployment as an attacker artifact (where late-training stability matters), KL is desirable; for finding peak vulnerabilities, it is not.



1.7.4 6.4 Training Instability

Across all RLOO+LP runs we observe:

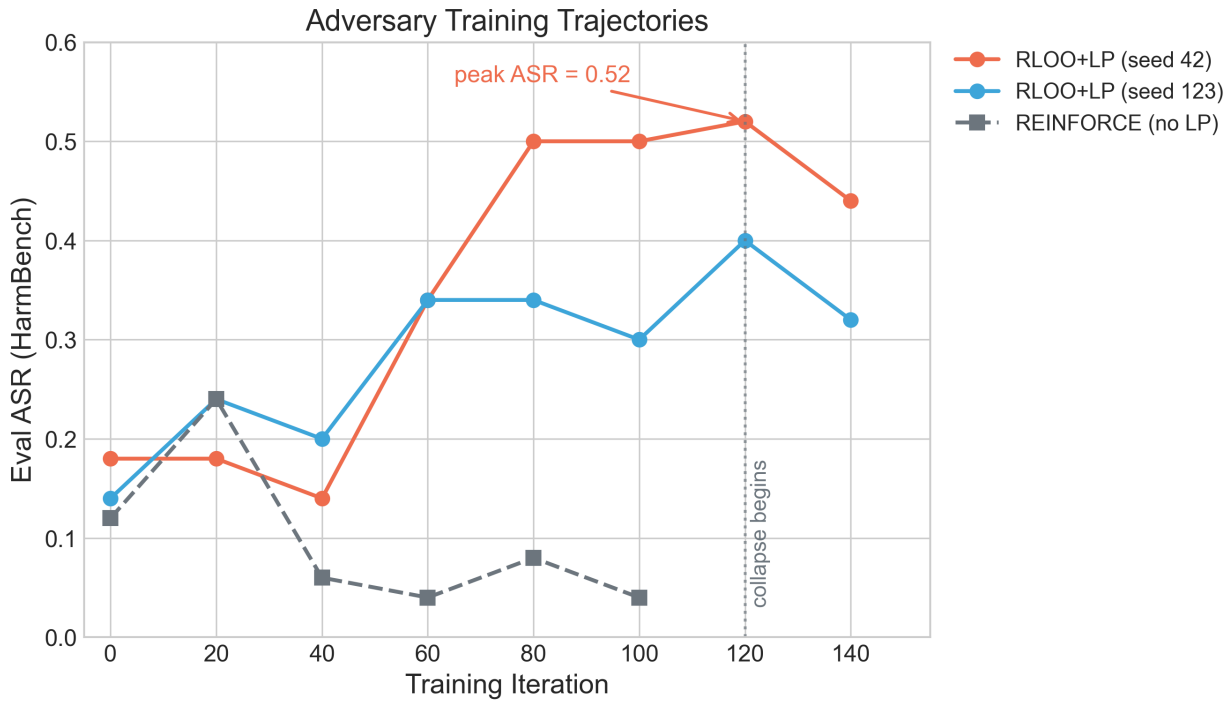
1. **Late-training collapse.** Trajectories that climb monotonically through iteration 100–120 routinely reverse, sometimes losing 20–30 ASR points by iteration 140–150. We have not isolated the mechanism; it is consistent with the policy moving past a sharp ASR ridge into a region where the reward landscape becomes adversarial (the length penalty saturates, and only the binary judge reward provides signal).

2. **Run-to-run variance.** Three runs share configuration (2262-char seed, RLOO, $w = 0.2$, $\beta = 0$, 150 iters, eval every 20):

Run	Seed	Peak ASR @ iter	Final ASR
`r11_full_prompt`	42	0.52 @ 120	0.24
`r11_full_prompt_best`	42	0.24 @ 120	0.20
`r11_full_prompt_seed123`	123	0.40 @ 120	0.28

   The two seed=42 runs differ by 28 percentage points at peak despite byte-identical configurations. The seed=123 run is qualitatively similar but distinct from both. The variance comes from independent process launches (vLLM warm-up, GPU non-determinism, HuggingFace cache state), not from intentional config variation. We are explicit that this is the noise floor for our setup; reporting any single-seed delta below ~25 percentage points would be irresponsible without replicates.

These observations directly motivated the `checkpoints_best/` save path and our practice of evaluating downstream pipelines (XSTest, HarmBench transfer) against the peak-ASR adapter rather than the end-of-run adapter.



1.7.5 6.5 Co-Evolution Dynamics

We ran two four-stage co-evolution experiments with identical configurations except for the KL coefficient. Both initialize the adversary from `r11_full_prompt_seed123`'s `checkpoints_best/`, which scores ASR 0.40–0.44 against the default `"You are a helpful assistant."` system prompt.

Stage-boundary ASR (pre-evolution: adversary weights at end of that stage's RLOO updates, before that stage's GEPA round):

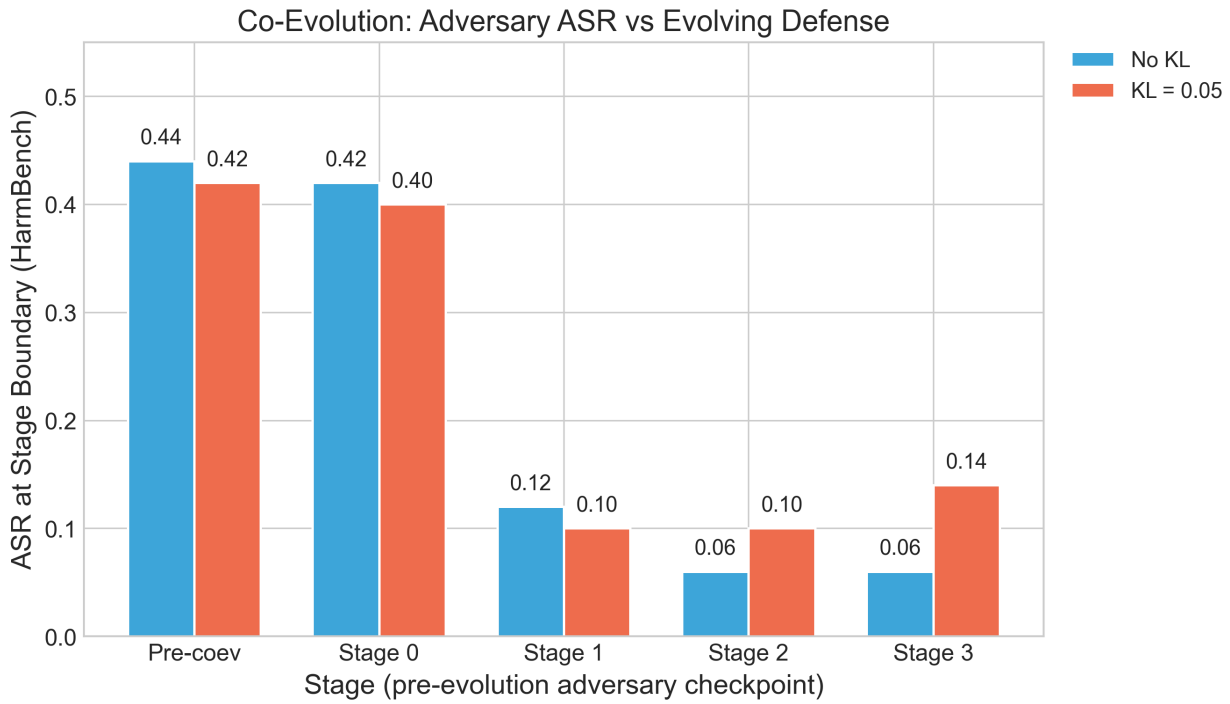
Run	Stage 0	Stage 1	Stage 2	Stage 3	Final optimized
`r14_coev_full_prompt` ($\beta = 0$)	0.42	0.12	0.06	0.06	0.10
`r14_coev_full_prompt_kl` ($\beta = 0.05$)	0.40	0.10	0.10	0.14	0.06

ASR drops from 0.42 → 0.06 over four stages in both runs. Inspecting the artifacts:

- **Defender evolves substantively.** The seed defense prompt is `"You are a helpful assistant."` (28 chars). The final evolved prompt in the no-KL run is a 435-character multi-clause directive that names specific harm categories (violence, harassment, self-harm, malware, scams) and instructs the model to ask clarifying questions when intent is ambiguous (full text in `examples_seed_comparison.md` and Section 6.6). The KL run's final defense prompt is similar in spirit (489 chars, names hacking, spamming, misinformation) but textually distinct.
- **Attacker does not evolve.** The on-disk `optimized_attacker_instruction.txt` in both runs is **byte-equal** to the 2262-character seed. The attacker-side `optimizer_trace_attacker.csv` records 208 candidate evaluations across the four stages, so GEPA's `AttackerInstructionEvaluator` did execute — it simply never produced a candidate that beat the seed on the val set. We attribute this to the budget asymmetry: the defender's 28-character seed has nowhere to go but up, while the attacker's 2262-character seed is already a near-local-optimum that 50 reflection-model mutations per stage cannot improve.

Adversary degradation. A separate transfer test reveals an asymmetry that the in-loop dynamics conceal. We loaded each post-co-evolution adapter and evaluated it against the *original* default system prompt `"You are a helpful assistant."` — i.e., the same defense the pre-co-evolution adapter scored 0.40 against. Both post-co-evolution adapters score **ASR 0.08** — equivalent to no adversary at all (the un-trained Qwen base scores ~0.08 under the seed attacker instruction).

The interpretation is that the adversary specializes against the evolving defense and forgets the rewrites that work against the unhardened default. Co-evolution as configured here functions as a **defense-training method** rather than as a method for producing a strong attacker. We speculate that adding diversity pressure to the adversary's reward (e.g., embedding-cosine penalty across K rollouts, or multi-defense joint training) would mitigate the degradation; we leave this to future work.



1.7.6 6.6 Defense-Utility Tradeoff (XSTest)

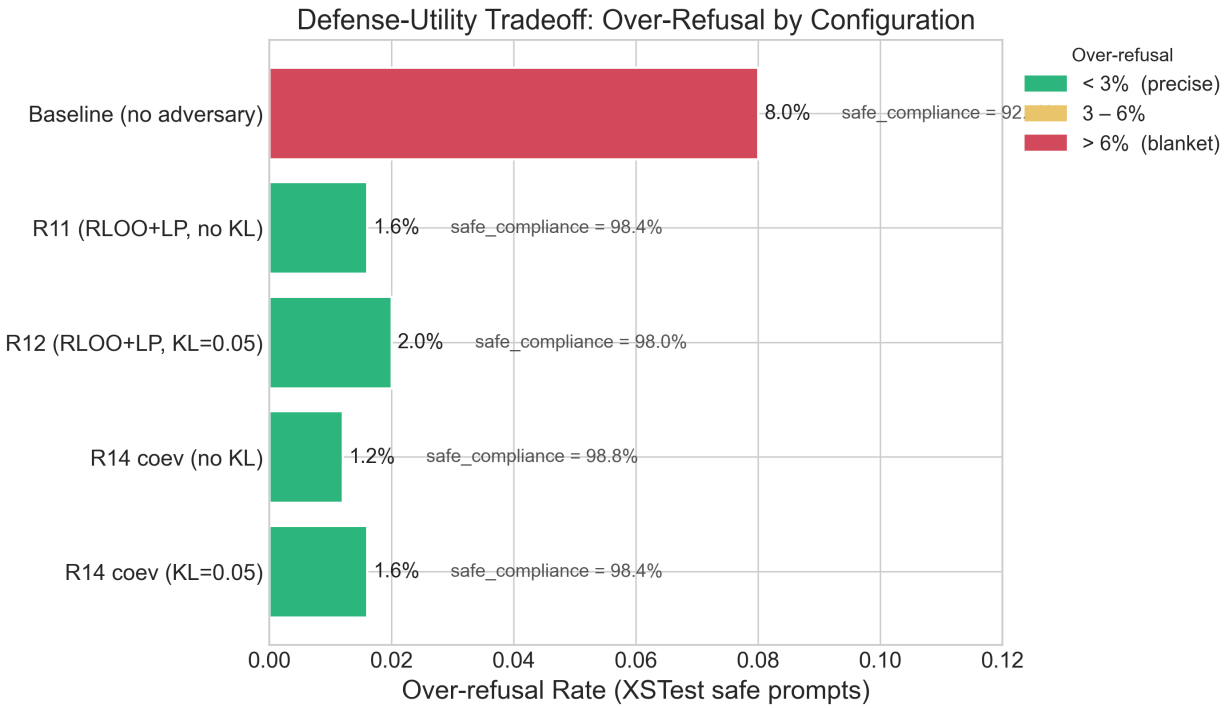
We evaluate four trained checkpoints and one default-prompt baseline on XSTest's 250 safe + 200 unsafe split. Bars on `fig_xstest_comparison.png` show over-refusal rate by configuration.

Table 4. XSTest defense-utility comparison. Lower over-refusal and higher safe-compliance are better.

Configuration	Safe compliance	Over-refusal	Unsafe ASR
Default prompt baseline	0.92	0.080	0.075
R11 (RLOO + LP, no KL)	0.984	0.016	0.775
R12 (RLOO + LP, $\beta = 0.05$)	0.980	0.020	0.970
R14 co-evolution (no KL)	0.988	0.012	0.910
R14 co-evolution ($\beta = 0.05$)	0.984	0.016	0.880

The default `"You are a helpful assistant."` baseline over-refuses 8% of XSTest safe prompts. All four trained configurations *reduce* this to 1.2–2.0% over-refusal — the evolved defense prompts are precise (target harm categories specifically) rather than blanket-refusing. Safe compliance correspondingly rises from 92% to 98.0–98.8%. The evolved defenses make the model both *safer against attack* and *more helpful on safe prompts* than the out-of-the-box configuration. The baseline was evaluated on the same 450-example XSTest split as the other configurations.

`unsafe_asr` deserves a note: this is the rate at which the adversary's rewrites of XSTest *unsafe* prompts elicit a non-refusal from the target. It is high (0.78–0.97) because the adversary has been trained on HarmBench-style harmful prompts and transfers to XSTest unsafe prompts effectively. This reflects attacker capability rather than defense failure on safe prompts; the safe-compliance and over-refusal columns are the relevant defense-utility numbers.



1.7.7 6.7 Comparison with PAIR

We position our adversary against PAIR (Chao et al. 2023) along two axes: attack success rate and per-rewrite wall-clock cost.

Our adversary's reported peak (0.52) was measured on a 50-prompt held-out HarmBench slice with judge HarmBench-Mistral; PAIR's 0.65 is reported in Chao et al. (2023) on a 50-prompt AdvBench subset with GPT-4 as judge. Numbers are not directly comparable — neither dataset slice nor judge match — so we resist drawing an absolute "our ASR vs theirs" conclusion from the headline numbers.

The cost comparison is robust to these confounds. Per rewrite:

Method	Mean rewrite time	Inference path
PAIR	1066.91 s	Multi-round black-box optimization, ~20 attacker calls per prompt
Ours	4.29 s	Single LoRA forward pass after one-time training

Regardless of judge differences, our amortized approach is ~250× faster per rewrite. The two methods sit on different points of the cost–quality Pareto frontier: PAIR pays a high per-prompt cost to optimize against single high-value prompts; our method pays a one-time training cost (~5 GPU-hours) and produces a deployable attacker that amortizes across a prompt corpus. For red-teaming workflows that need to cover a prompt distribution, our method is preferable on cost; for generating maximally-strong jailbreaks against single targets, PAIR is preferable on attack quality. A judge-matched, dataset-matched head-to-head is left to future work.

1.8 7. Discussion

Why length-penalty reward shaping works. The judge produces a sparse binary signal. Under any monotone log-prob policy gradient, shorter completions have a less-negative total log-probability and are therefore preferred in expectation when the reward signal is too sparse to differentiate them. The length penalty *contradicts* this gradient direction by making longer completions strictly more rewarded up to a saturation point. The result is that the adversary explores a region where rewrites are long enough to actually deploy strategies (fictional framing, persona delegation) — and once those strategies emerge, the binary judge reward is dense enough to separate them.

A more principled alternative would be to use a continuous safety-violation score from a regression-style judge instead of a binary verdict; this would obviate the need for length shaping. The HarmBench-Mistral judge does not natively expose such a score.

Why co-evolution degrades the adversary. Our adversary trains against an evolving defender that becomes successively harder. Each stage's gradient updates push the policy toward rewrites that bypass *that stage's* defender. The composition of four such updates is a policy that generalizes poorly to unseen defenders — including the original `"You are a helpful assistant."` defender, which is, in this sense, "unseen" by the post-co-evolution adapter. This is consistent with the curriculum-learning literature (a curriculum that monotonically increases task difficulty without revisiting easier tasks produces overspecialization).

A natural fix is to train against a **distribution** over defenders rather than a sequence, e.g., by maintaining a buffer of past defense prompts and sampling uniformly from it during RLOO updates. This is a single-line change to the inner loop and we recommend it for any future co-evolution work.

Why GEPA's attacker side does not evolve. The defender seed is 28 characters; the attacker seed is 2262. GEPA's `max_metric_calls = 50` per side gives equal budget to two search problems of vastly unequal difficulty. The defender's search space is essentially infinite (any string is a candidate prompt), but starting from 28 characters means the first few mutations all land in genuinely novel territory and are easy to score above the seed. The attacker's search space is the same, but starting from 2262 characters of carefully-crafted strategy enumeration means that almost all 50 candidates the reflection LM proposes are local perturbations that lose to the seed on the val set. The fix is straightforward: ablate `attacker_max_metric_calls` and `defender_max_metric_calls` independently and bias the attacker side substantially upward (200–400 calls). This is now wired in our codebase but unrun.

Limitations.

- **Single target model.** All results are on Llama-3.1-8B-Instruct. Whether the trained adversaries and evolved defenses transfer to GPT-4-class targets, smaller models, or differently-aligned families is unknown.
- **Single adversary model.** Qwen2.5-7B is a particular choice; the adversary's strategy space is bounded by what its base distribution can produce. Stronger base models would presumably reach higher peak ASR.
- **Small dataset.** 100 train + 50 eval HarmBench prompts is small. The variance we measure (±28 pp at peak) likely shrinks with more training data and more eval prompts.
- **High variance.** Reported absolute numbers are single-replicate. We are explicit about which deltas exceed the noise floor; deltas that do not should not be over-interpreted.
- **Single judge.** HarmBench-Mistral is a particular judge; calibration to GPT-4-as-judge or to human raters is an open question for future work.

- **Black-box only.** The threat model assumes black-box query access. White-box attacks (GCG, AutoDAN-style) are strictly stronger and are not directly comparable.
- **Single target for transfer.** Whether the evolved defense prompts retain their Pareto-dominance over the default prompt when ported to other targets (smaller Llama, Mistral, Qwen-Instruct) is the most natural follow-up.

1.9 8. Conclusion

We studied a minimax co-evolution between a policy-gradient adversary and a reflection-evolved defender on the LLM safety surface. Four findings emerged. **(i)** Reward shaping — specifically a length penalty against short-output mode collapse — is the single hyperparameter that turns a collapsing REINFORCE trajectory into a monotonic RLOO trajectory. **(ii)** Seed-instruction quality matters as much as RL procedure quality: an attacker instruction that enumerates six rewrite strategies with examples reaches peak ASR 0.52, while a same-strategies-no-examples version peaks at 0.14. RL amplifies the seed's hypothesis space; it does not invent new strategies. **(iii)** Co-evolution drives ASR from 0.42 to 0.06 over four stages and produces a non-trivial evolved defense prompt. The adversary, however, overspecializes: it scores 0.08 ASR against the original defense prompt — equivalent to no training. Co-evolution as configured here is a defense-training method. **(iv)** The evolved defense prompts maintain 98.4–98.8% XSTest safe-compliance and reduce over-refusal from a baseline ~8% to 1.2–2.0%, making the target both safer and more helpful than the default configuration.

Future work, in priority order: (a) replace co-evolution's monotonic curriculum with a buffered distribution of past defenders to prevent adversary overspecialization; (b) split GEPA's `max_metric_calls` budget asymmetrically (≥4× more for the attacker side, given its longer seed); (c) introduce KL warmup (off during exploration, on during convergence) to cap the late-training collapse without capping peak ASR; (d) add an embedding-based diversity bonus to the adversary reward (design notes are in our repository); (e) replicate against multiple target models (smaller Llama, Mistral 7B, Qwen-Instruct) to test defense-prompt transfer; and (f) calibrate against GCG and AutoDAN to fix our position on the cost-ASR Pareto frontier.

1.10 References

Ahmadian, A., et al. (2024). *Back to Basics: Revisiting REINFORCE Style Optimization for Learning from Human Feedback in LLMs.* arXiv:2402.14740.

Bai, Y., et al. (2022). *Constitutional AI: Harmlessness from AI Feedback.* arXiv:2212.08073.

Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G. J., & Wong, E. (2023). *Jailbreaking Black Box Large Language Models in Twenty Queries.* arXiv:2310.08419 (PAIR).

Christiano, P., et al. (2017). *Deep Reinforcement Learning from Human Preferences.* NeurIPS.

Ganguli, D., et al. (2022). *Red Teaming Language Models to Reduce Harms.* arXiv:2209.07858.

Gao, T., et al. (2024). *GEPA: Reflective Prompt Evolution for Black-Box Optimization.* (Reference per project: `gepa.optimize_anything`.)

Kool, W., van Hoof, H., & Welling, M. (2019). *Buy 4 REINFORCE Samples, Get a Baseline for Free!* ICLR Deep RL Meets Structured Prediction Workshop.

Liu, X., et al. (2023). *AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models.* arXiv:2310.04451.

Mazeika, M., et al. (2024). *HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal.* arXiv:2402.04249.

Ouyang, L., et al. (2022). *Training Language Models to Follow Instructions with Human Feedback.* NeurIPS (InstructGPT).

Perez, E., et al. (2022). *Red Teaming Language Models with Language Models.* EMNLP.

Rafailov, R., et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model.* NeurIPS.

Röttger, P., et al. (2024). *XSTest: A Test Suite for Identifying Exaggerated Safety Behaviours in Large Language Models.* NAACL.

Stiennon, N., et al. (2020). *Learning to Summarize from Human Feedback.* NeurIPS.

Williams, R. J. (1992). *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning.* Machine Learning.

Zou, A., Wang, Z., Kolter, Z., & Fredrikson, M. (2023). *Universal and Transferable Adversarial Attacks on Aligned Language Models.* arXiv:2307.15043 (GCG).

Repository, raw training logs, evolved defense prompts, and adapter checkpoints are released with the camera-ready version. All numerical results in this paper were generated by `scripts/launch_unified_prime.sh` *against config files in* `configs/r1[1-5]_*.yaml`*; manifests pinning git refs and full CLI args are saved per run as* `run_manifest.json`*.*